

Chapter 1 - Laboratory Activity

Laboratory Title

Establishing a Professional TypeScript, GitHub, and AI-Assisted Development Workflow

Laboratory Objectives

At the end of this laboratory activity, students should be able to:

1. configure a basic modern web-development environment;
 2. create and initialise a Node.js and TypeScript project;
 3. configure Git source control;
 4. construct an appropriate `.gitignore`;
 5. configure linting and formatting;
 6. create and manage a GitHub repository;
 7. create a GitHub Issue;
 8. develop a feature using a feature branch;
 9. use an AI coding assistant responsibly;
 10. document accepted and rejected AI recommendations;
 11. verify AI-generated TypeScript code;
 12. create meaningful Git commits;
 13. open a Pull Request; and
 14. demonstrate a professional development workflow.
-

Scenario

You are a junior developer assigned to begin the development of a **University Student Services Portal**.

The first task is not to build the complete system.

Your responsibility is to establish a professional project environment that the development team can use for succeeding features.

Your repository must support:

- TypeScript;
- version control;

- GitHub collaboration;
 - linting;
 - formatting;
 - feature branches;
 - Pull Requests;
 - and responsible use of an AI coding assistant.
-

Laboratory Requirements

Students need access to:

- Git;
 - GitHub account;
 - Visual Studio Code or another approved editor;
 - Node.js;
 - npm or pnpm;
 - TypeScript;
 - ESLint;
 - Prettier;
 - web browser;
 - and an instructor-approved AI coding assistant, if available.
-

Part 1 – Verify the Development Environment

Open a terminal and verify the required software.

Example:

```
git --version
node --version
npm --version
```

If pnpm is used:

```
pnpm --version
```

Record the versions in your laboratory report.

Part 2 – Create the Project Directory

Create a directory:

```
mkdir student-services-portal  
cd student-services-portal
```

Open the directory using your editor.

Part 3 – Initialise the Node.js Project

Using npm:

```
npm init -y
```

Inspect the generated:

```
package.json
```

Identify:

- project name;
 - version;
 - scripts;
 - dependencies;
 - and development dependencies.
-

Part 4 – Install TypeScript

Install TypeScript as a development dependency according to the current official TypeScript setup guidance.

Create a TypeScript configuration file.

The project should eventually contain:

tsconfig.json

Create:

src/

and inside it:

index.ts

Part 5 – Create the Initial TypeScript Program

Add the following model:

```
interface Student {  
  id: number;  
  name: string;  
  email: string;  
  status: "active" | "inactive";  
}
```

Create a function:

```
function formatStudent(student: Student): string {  
  return `${student.id} - ${student.name} (${student.status})`;  
}
```

Create a sample student and display the result.

Compile or execute the project using the approved TypeScript development workflow.

Part 6 – Add a Generic API Response Type

Create:

```
interface ApiResponse<T> {  
    success: boolean;  
    data: T;  
}
```

Use it to represent:

```
ApiResponse<Student>
```

and:

```
ApiResponse<Student[]>
```

Explain in your laboratory report why the generic version is preferable to:

```
interface ApiResponse {  
    success: boolean;  
    data: any;  
}
```

Part 7 – Add Runtime Validation

Create a function that accepts:

```
unknown
```

and verifies whether it satisfies the required `Student` structure.

Demonstrate at least:

- one valid object;

- one invalid object containing an incorrect `id`;
- and one invalid object missing the student's name.

Explain why the TypeScript interface alone cannot guarantee that external API data is valid.

Part 8 – Configure ESLint and Prettier

Configure:

- linting;
- formatting;
- appropriate npm scripts.

The project should provide commands conceptually equivalent to:

```
npm run lint  
npm run format
```

Run both tools.

Include screenshots or terminal results in your laboratory documentation.

Part 9 – Create `.gitignore`

Create:

```
.gitignore
```

It should include appropriate entries such as:

```
node_modules/  
dist/  
.env  
*.log
```

Explain the reason for each ignored pattern.

Part 10 – Initialise Git

Run:

```
git init  
git status
```

Review all files before staging them.

Do not stage secrets or inappropriate generated files.

Part 11 – Create the First Commit

Stage the approved files:

```
git add .
```

Review:

```
git status
```

Create a meaningful initial commit.

Example:

```
git commit -m "chore: initialize TypeScript project"
```

Part 12 – Create the GitHub Repository

Create a GitHub repository for the project.

Connect the local project to the remote repository.

Push the initial branch.

Verify that:

- source code appears on GitHub;
 - `node_modules/` does not appear;
 - `.env` does not appear;
 - README is visible;
 - and commit history is present.
-

Part 13 – Create a README

Your README must contain:

Project Title

Project Description

Requirements

Installation Instructions

How to Run the Project

How to Run Linting

How to Format Code

Development Workflow

AI Usage Policy

Suggested AI policy:

AI tools may be used to assist development.
All AI-generated code must be reviewed, modified
when necessary, tested, and verified before commit.

Part 14 – Create a GitHub Issue

Create the Issue:

Add student-status formatter

Provide:

Description

The application should return a user-friendly label for the student's status.

Acceptance Criteria

"active" → "Active Student"

"inactive" → "Inactive Student"

Invalid or unexpected values should be handled safely.

Part 15 – Create a Feature Branch

Create:

`git switch -c feature/student-status`

Verify:

`git branch`

All feature development must occur on this branch.

Part 16 – AI-Assisted Development Exercise

Use an instructor-approved AI coding assistant.

Ask the AI to:

Suggest a TypeScript implementation for converting a student's active/inactive status into a readable label.

Explain the implementation and include possible edge cases. Do not use the any type.

Save the AI response in your laboratory documentation.

Do **not** immediately copy and commit it.

Part 17 – AI Review Form

Complete the following:

AI Tool:

Prompt Used:

AI Recommendation:

What I Understood:

Recommendation Accepted:

Recommendation Modified:

Recommendation Rejected:

Reason:

Part 18 – Implement the Feature

Create your final implementation.

Example structure:

```
type StudentStatus =  
  | "active"  
  | "inactive";  
  
function getStudentStatusLabel(  
  status: StudentStatus  
): string {  
  if (status === "active") {  
    return "Active Student";  
  }  
  
  return "Inactive Student";  
}
```

Students may use another correct implementation.

The important requirement is that the student can explain it.

Part 19 – Test the Feature

Verify:

active
inactive

Run:

- TypeScript checks;
- linting;
- formatting;
- and available tests.

If the AI suggested an implementation that fails a test or violates requirements, document this finding.

Part 20 – Verify the AI Recommendation

Students must verify any unfamiliar syntax, library, or API using official documentation.

Record:

Claim or Code Verified:

Source:

Result:

Part 21 – Commit the Feature

Review:

```
git status  
git diff
```

Commit the verified implementation.

Example:

```
git add .  
git commit -m "feat: add student status formatter"
```

Part 22 – Push the Feature Branch

Push:

```
git push -u origin feature/student-status
```

Part 23 – Open a Pull Request

Create a Pull Request.

The PR description must contain:

Summary

What Changed

Testing Performed

AI Usage

Known Limitations

Related Issue

Link the Pull Request to the corresponding GitHub Issue.

Part 24 – Perform Code Review

Another student or the instructor should review the Pull Request.

The reviewer should examine:

- correctness;

- naming;
- TypeScript types;
- runtime safety;
- linting;
- formatting;
- unnecessary complexity;
- AI-generated assumptions;
- and documentation.

At least one meaningful review comment should be provided.

Part 25 – Revise if Necessary

If the reviewer identifies an issue:

1. modify the code on the feature branch;
 2. test the fix;
 3. commit the change;
 4. push again;
 5. and observe how the Pull Request is automatically updated.
-

Part 26 – Merge the Pull Request

After approval, merge the Pull Request into the main branch.

Verify:

- the Issue is completed or closed;
 - the Pull Request is merged;
 - the main branch contains the feature;
 - and repository history remains understandable.
-

Laboratory Deliverables

Submit the following:

1. GitHub repository URL;
 2. screenshot of development environment;
 3. `README.md`;
 4. `.gitignore`;
 5. `package.json`;
 6. TypeScript configuration;
 7. ESLint configuration;
 8. Prettier configuration;
 9. TypeScript source files;
 10. runtime-validation example;
 11. GitHub Issue;
 12. feature branch;
 13. meaningful commit history;
 14. Pull Request;
 15. code-review evidence;
 16. AI prompt and response;
 17. AI review form;
 18. explanation of one AI suggestion that was accepted, modified, or rejected;
 19. verification source used;
 20. short reflection.
-

Laboratory Reflection Questions

Answer each question in three to five sentences.

1. What was the most important difference between your previous programming workflow and the Git/GitHub workflow used in this laboratory?
2. Why was the feature branch useful?
3. Did the AI provide any suggestion that required modification? Explain.
4. How did TypeScript help detect or prevent a possible problem?
5. Why was runtime validation still necessary?
6. What information should never be placed in the repository?
7. Which step of:

Ask → Understand → Review → Modify → Test → Verify → Commit

was the most important to you? Explain your answer.

8. How could this workflow improve a group software-development project?
-

Assessment Rubric

Total: 100 Points

Criterion	Excellent	Proficient	Developing	Beginning	Points
Development Environment	All required tools are properly configured and demonstrated.	Most tools are properly configured with minor issues.	Several configuration issues are present.	Environment is incomplete or unusable.	10
TypeScript Implementation	Demonstrates strong typing, interfaces, generics, narrowing, and appropriate use of <code>unknown</code> ; code is clear and correct.	Most TypeScript concepts are correctly applied.	TypeScript is used but with conceptual or implementation weaknesses.	TypeScript concepts are largely incorrect or missing.	15
Runtime Validation	Correctly validates external/unknown data and clearly explains the compile-time/runtime distinction.	Validation is generally correct with minor gaps.	Validation is incomplete or weakly explained.	Runtime validation is missing or incorrect.	10
Git Practices	Repository, commits, branch workflow, <code>.gitignore</code> , and history demonstrate professional practice.	Git workflow is generally correct with minor issues.	Several Git practices are inconsistent.	Little evidence of appropriate Git use.	15

GitHub Collaboration	Issue, feature branch, Pull Request, linkages, and review process are complete and clear.	Most collaboration requirements are present.	Some GitHub workflow components are missing.	Pull Request and collaboration workflow are largely absent.	15
ESLint and Prettier	Both are correctly configured, executed, and integrated into project scripts.	Both are configured with minor issues.	Only partial quality-tool configuration is demonstrated.	Quality tools are absent or unusable.	10
Responsible AI Use	AI usage is fully documented; student demonstrates understanding, verification, testing, and critical evaluation of suggestions.	AI usage is appropriately documented and mostly verified.	AI was used but review or verification is weak.	AI-generated work appears to have been accepted without understanding or verification.	15
Documentation	README, AI disclosure, technical notes, and instructions are complete and professional.	Documentation is generally complete.	Documentation contains notable omissions.	Documentation is incomplete or unclear.	5
Code Quality and Professionalism	Code is readable, well organised, maintainable, and follows project conventions.	Code quality is generally good.	Code contains several maintainability or organisation issues.	Code quality is poor or difficult to understand.	5

TOTAL: 100 POINTS